

LTNtorch: PyTorch Implementation of Logic Tensor Networks

Tommaso Carraro^{1,2}

TCARRARO@FBK.EU

Luciano Serafini²

SERAFINI@FBK.EU

Fabio Aiolli¹

AIOLLI@MATH.UNIPD.IT

¹ *Department of Mathematics, University of Padova, Padova, PD 35121, IT*

² *Data and Knowledge Management, Fondazione Bruno Kessler, Povo, TN 38123, IT*

Editor: My editor

Abstract

Logic Tensor Networks (LTN) is a Neuro-Symbolic framework that effectively incorporates deep learning and logical reasoning. In particular, LTN allows defining a logical knowledge base and using it as the objective of a neural model. This makes learning by logical reasoning possible as the parameters of the model are optimized by minimizing a loss function composed of a set of logical formulas expressing facts about the learning task. The framework learns via gradient-descent optimization. Fuzzy logic, a relaxation of classical logic permitting continuous truth values in the interval $[0, 1]$, makes this learning possible. Specifically, the training of an LTN consists of three steps. Firstly, (i) the training data is used to ground the formulas. Then, (ii) the formulas are evaluated, and the loss function is computed. Lastly, (iii) the gradients are back-propagated through the logical computational graph, and the weights of the neural model are changed so the knowledge base is maximally satisfied. LTNtorch¹ is the fully documented and tested PyTorch implementation of Logic Tensor Networks. This paper presents the formalization of LTN and how LTNtorch implements it. Moreover, it provides a basic binary classification example.

Keywords: neuro-symbolic integration, logic tensor networks, framework

1 Introduction

Neuro-Symbolic integration (NeSy) (d’Avila Garcez et al., 2009) is the branch of Artificial Intelligence that studies the incorporation of symbolic AI (e.g., logical reasoning) and sub-symbolic AI (e.g., deep learning). The idea of NeSy is to build systems that can merge the advantages of these two paradigms. For example, neural networks effectively learn in the presence of noise; however, they require a lot of samples and are usually black boxes. On the other hand, symbolic AI can perform few-shot learning and, in some cases, even zero-shot learning. Plus, it is usually explainable by design as it is often implemented as a set of logical formulas. An effective integration could address the major limitations of neural networks. Logic Tensor Networks (LTN) (Badreddine et al., 2022) take a step in this direction by proposing a framework to learn neural networks using logical reasoning. In particular, LTN allows for the specification of logical loss functions and their minimization by gradient-descent optimization. A specific First-Order language, plus the use of fuzzy

1. Link to publicly available repository: <https://github.com/tommasocarraro/LTNtorch>.

logic semantics (Cintula et al., 2011) to implement logical operators, makes this learning possible. LTN was originally implemented in TensorFlow². This paper presents the PyTorch implementation, dubbed LTNtorch.

2 Framework formalization

To define the knowledge base, LTN uses a specific first-order language called Real Logic. It is fully differentiable and has concrete semantics that allow mapping every symbolic expression into the domain of real numbers. This mapping function is referred to as *grounding*, denoted by $\mathcal{G}$. Thanks to $\mathcal{G}$, LTN can convert logical formulas into computational graphs that enable gradient-based optimization based on fuzzy logic semantics. Real Logic is a first-order (Flach, 2007) language with a signature containing constant, variable, functional, and predicate symbols. A term is constructed recursively from constants, variables, and functional symbols. An expression formed by applying a predicate symbol to some term(s) is called an atomic formula. Complex formulas are constructed recursively using connectives (i.e., $\neg, \wedge, \vee, \implies, \leftrightarrow$) and quantifiers (i.e., $\forall, \exists$).

2.1 LTN grounding

$\mathcal{G}$ maps each individual into a tensor of real features (e.g., sample’s features), functions as real functions (e.g., regression functions), and predicates as real functions that specifically project onto a value in the interval $[0, 1]$ (e.g., classifiers). A variable x is grounded to a *sequence* of n_x individuals from a domain (see Fig. 1(a)), with $n_x \in \mathbb{N}^+, n_x > 0$. As a consequence, a term $t(x)$ or a formula $P(x)$, constructed recursively with variable x , will be grounded to a sequence of n_x values too (see Fig. 1(b, c)). Afterward, connectives are grounded using fuzzy semantics, while quantifiers using special aggregation functions. LTN provides different fuzzy semantics for these operators, namely *Product*, *Gödel*, and *Lukasiewicz* semantics (Cintula et al., 2011). In what follows, we define the *Product* configuration, better suited for gradient-based optimization (van Krieken et al., 2022). In the formalization, $u, v, u_1, \dots, u_n \in [0, 1]$, and $p \geq 1$.

$$\begin{aligned} \mathcal{G}(\wedge) : u, v &\mapsto u \cdot v & \mathcal{G}(\vee) : u, v &\mapsto u + v - u \cdot v & \mathcal{G}(\implies) : u, v &\mapsto 1 - u + u \cdot v & \mathcal{G}(\neg) : u &\mapsto 1 - u \\ \mathcal{G}(\exists) : u_1, \dots, u_n &\mapsto \left(\frac{1}{n} \sum_{i=1}^n (u_i)^p\right)^{\frac{1}{p}} & \mathcal{G}(\forall) : u_1, \dots, u_n &\mapsto 1 - \left(\frac{1}{n} \sum_{i=1}^n (1 - u_i)^p\right)^{\frac{1}{p}} \end{aligned}$$

Connective operators are applied element-wise to the tensors in input (see Fig. 1(d)), while aggregators aggregate the dimension of the tensor in input that corresponds to the quantified variable (see Fig. 1(e, f)). Figure 1 gives an intuition of how LTN performs these operations to compute the formula $\forall x \exists y P(x, y) \wedge Q(y)$, where P and Q are logical predicates, while x and y are variables. Note these are also the steps LTNtorch uses to construct the computational graph for this formula.

2.2 LTN learning

Given a Real Logic knowledge base $\mathcal{K} = \{\phi_1, \dots, \phi_n\}$, where $\phi_1, \dots, \phi_n$ are closed formulas, LTN allows to learn the grounding of constants, functions, and predicates appearing in them. In particular, if constants are grounded as embeddings, and functions/predicates onto neural networks,

2. First implementation of LTN: <https://github.com/logictensornetworks/logictensornetworks>.

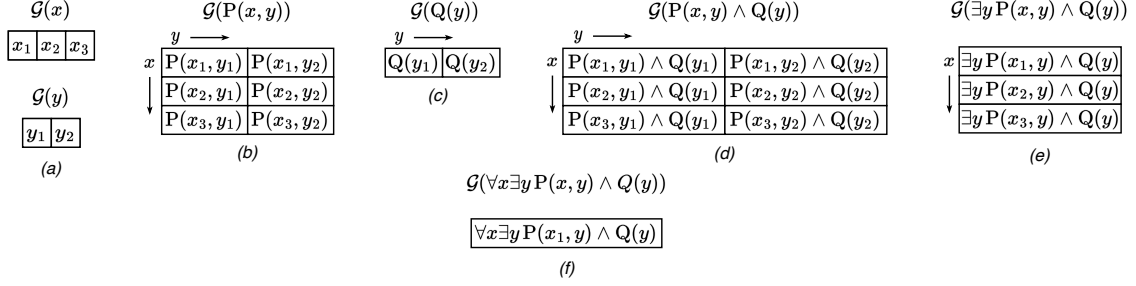


Figure 1: Step by step computation of $\forall x \exists y P(x, y) \wedge Q(y)$ in LTN. (a) Variables x and y are grounded as sequences of 3 and 2 individuals, respectively. Note each individual (e.g., x_1) is a real-valued tensor whose dimensions are determined by the dataset (e.g., image, vector, or scalar). (b) Computation of binary predicate P . Each item in this grid is a truth value in $[0, 1]$. Note how LTN organizes the output in two dimensions, one for x and one for y . $P(x_1, y_1)$ is the evaluation of P on x_1 and y_1 . (c) Computation of unary predicate Q , similar to P . (d) Computation of $\wedge$ applying $\mathcal{G}(\wedge)$ element-wise. Note that broadcasting is used to match the dimensions of $\mathcal{G}(P(x, y))$ and $\mathcal{G}(Q(y))$ before applying element-wise multiplication. (e) Computation of $\exists$ by aggregating y 's dimension using $\mathcal{G}(\exists)$. (f) Computation of $\forall$ by aggregating x 's dimension using $\mathcal{G}(\forall)$.

their grounding $\mathcal{G}$ depends on some learnable parameters θ . We denote a parametric grounding as $\mathcal{G}(\cdot|\theta)$. In LTN, the learning of parametric groundings is obtained by finding parameters θ^* that maximize the satisfaction of $\mathcal{K}$, namely $\theta^* = \operatorname{argmax}_{\theta} \operatorname{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}(\phi|\theta)$. In the notation, $\operatorname{SatAgg} : [0, 1]^* \mapsto [0, 1]$ is a formula aggregating operator, often defined using $\mathcal{G}(\forall)$. It represents the overall satisfaction of the knowledge base $\mathcal{K}$. As the objective is to maximize the overall satisfaction of $\mathcal{K}$, LTNTorch minimizes $1 - \operatorname{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}(\phi|\theta)$. Because Real Logic grounds expressions in continuous domains, LTN attaches gradients to every sub-expression and consequently learns through gradient-descent optimization.

3 Binary classification in LTNTorch

We introduce a simple binary classification task where the objective is to discriminate between cats and dogs. The building blocks for this example are (1) a variable *dog* to iterate through batches of dog images, (2) a variable *cat* to iterate through batches of cat images, and (3) a unary predicate *Dog* distinguishing between cats and dogs. Given these symbols, the knowledge base LTNTorch maximizes to solve this task is $\mathcal{K} = \{\phi_1, \phi_2\}$, where $\phi_1 = \forall \text{dog } \text{Dog}(\text{dog})$ and $\phi_2 = \forall \text{cat } \neg \text{Dog}(\text{cat})$. When $\mathcal{K}$ is maximally satisfied, *Dog* returns high truth values when dogs are given and low truth values (see the use of $\neg$ in ϕ_2) when cats are given.

LTNTorch makes use of $\mathcal{G}$ to build the computational graph for this task. In particular, $\mathcal{G}(\text{dog}) = \mathbb{R}^{b \times l \times w \times c}$, where b is the batch dimension, and l , w , and c are the length, width, and number of channels of the image, respectively. In other words, *dog* is grounded as a batch of dog images. Similarly, *cat* is grounded as a batch of cat images (see Fig. 2). Then, $\mathcal{G}(\text{Dog}|\theta) : \mathbf{x} \mapsto \sigma(\text{CNN}_{\theta}(\mathbf{x}))$, namely *Dog* is implemented through a Convolutional Neural Network (i.e., CNN) with parameters θ . Specifically, this predicate takes images in input and returns truth values indicating how plausible it is that the images depict dogs (see Fig. 2). Note the logistic function σ allows *Dog* to be interpreted as a fuzzy logic predicate, as the output is in the interval $[0, 1]$. The loss function for this task is:

$$\mathcal{L}(\theta) = 1 - \operatorname{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_{\text{dog} \leftarrow \mathcal{B}_{\text{dog}}, \text{cat} \leftarrow \mathcal{B}_{\text{cat}}}(\phi|\theta)$$